GENERATIVE AI COURSE

Gen AI Course in 20 Days

with 7+ Hands-On Projects

Deep Learning • Attention • Transformers • LLM Fundamentals • RAG

A practical, industry-oriented program that moves from neural network foundations to modern LLM applications, combining concepts, hands-on implementation and project work.

20	40	3	7+
Days of live classes	Live hours (2 hrs/day)	Learning phases	Hands-on projects



1. Phase 1 — Deep Learning Foundations

Phase 1 builds the core of how neural networks learn from data and process images and sequences. Every later phase depends on these ideas.

A. Artificial Neural Networks (ANN)

TOPICS COVERED

- Neural network fundamentals
- Neurons, weights, biases and activation functions
- Forward propagation
- Loss functions
- Backpropagation
- Gradient descent and optimization
- Training, validation and testing
- Overfitting and regularization

Why it is taught: To establish the fundamentals of learning from data and understand how neural networks make predictions and improve through training.

B. Convolutional Neural Networks (CNN)

TOPICS COVERED

- Image representation and preprocessing
- Convolution layers
- Filters and feature maps
- Pooling layers
- Flattening and fully connected layers
- CNN training and evaluation
- Image classification workflow

Why it is taught: To understand how deep learning models extract visual features and solve image classification and computer vision problems.

C. Recurrent Neural Networks (RNN)

TOPICS COVERED

- Sequential data
- RNN architecture
- Hidden states
- Sequence prediction
- Vanishing gradient problem
- Practical sequence-modelling workflow

Why it is taught: To introduce models that process ordered data such as text, time series and other sequences.

D. LSTM and GRU

TOPICS COVERED

- Limitations of basic RNNs
- LSTM gates and cell states
- GRU architecture
- Sequence modelling
- Time-series and text prediction applications

Why it is taught: To explain how improved recurrent architectures retain useful information across longer sequences.

E. NLP Foundations and Sequence-to-Sequence Models

TOPICS COVERED

- Text preprocessing
- Tokenization
- Vocabulary and integer encoding
- Padding and sequence preparation
- Embeddings
- Encoder–Decoder architecture
- Sequence-to-Sequence learning
- Teacher forcing and inference fundamentals

Why it is taught: To connect deep learning with language processing and prepare students to understand attention mechanisms and modern language models.

2. Phase 2 — Attention, Transformers and Large Language Models

Phase 2 moves from recurrent models to attention, the idea behind Transformers and today's LLMs, and shows how LLMs are used in applications.

A. Attention Mechanisms

TOPICS COVERED

- Why attention is needed
- Query, Key and Value (Q, K, V)
- Attention scores
- Scaled dot-product attention
- Self-attention
- Multi-head attention
- Causal and masked attention

Why it is taught: To help students understand how a model identifies relevant information in a sequence instead of relying only on recurrent processing.

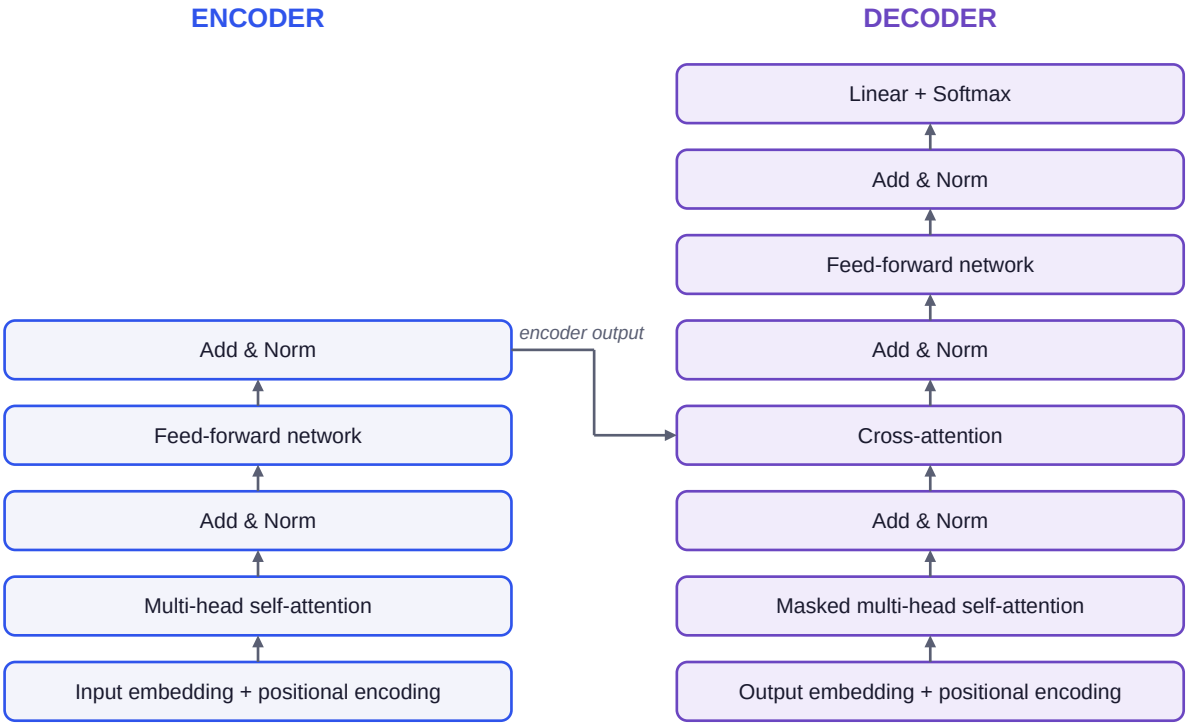
B. Transformer Architecture

TOPICS COVERED

- Transformer overview
- Encoder and decoder blocks
- Self-attention and cross-attention
- Positional encoding
- Feed-forward networks
- Residual connections
- Layer normalization
- Transformer training and inference flow
- Encoder-only and decoder-only models
- BERT architecture and use cases

Why it is taught: To build a clear understanding of the architecture behind modern NLP systems and many large language models.

Transformer architecture — simplified view



Simplified diagram of the original encoder–decoder Transformer. Encoder-only models (such as BERT) use the encoder side; decoder-only models use a decoder-style stack without cross-attention.

C. Large Language Model (LLM) Fundamentals

TOPICS COVERED

- What an LLM is
- Tokens versus words
- Tokenization
- Embeddings
- Context windows
- Pretraining
- Fine-tuning and instruction tuning
- Human-feedback-based alignment and preference optimization
- Temperature, Top-K and Top-P sampling
- Structured outputs
- Hallucinations and limitations
- API authentication and integration
- System, user and assistant messages

Why it is taught: To explain how LLMs process inputs, generate responses and can be integrated into practical applications.

Training, fine-tuning and inference

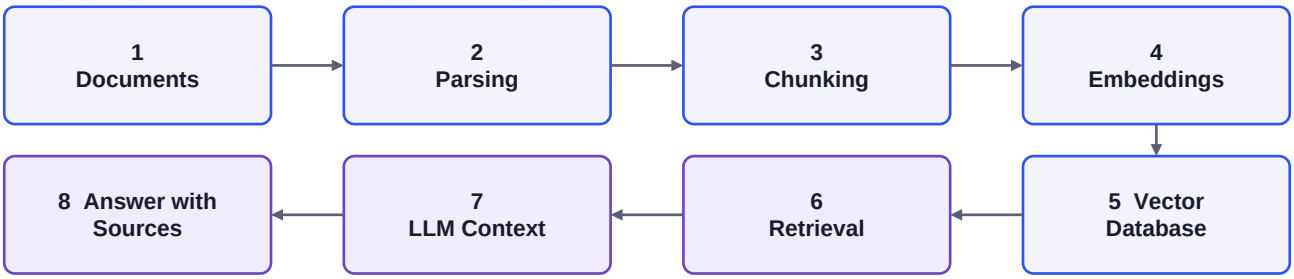
Stage	What happens	Role in this course
Training (pretraining)	A model learns general language patterns from very large text datasets, using large-scale compute.	Studied conceptually. Students do not train a massive LLM from scratch.
Fine-tuning	A pretrained model is further adapted to a task, domain or instruction style using a smaller dataset.	Studied conceptually, including instruction tuning and preference optimization.
Inference	A trained model generates output for a given prompt, controlled by settings such as temperature, Top-K and Top-P.	Hands-on focus: calling LLMs through APIs and building applications.

3. Phase 3 — Retrieval-Augmented Generation and LLM Applications

Phase 3 shows how to make LLMs answer questions from specific documents and knowledge sources, with traceable sources. **Why this phase is taught:** to help students build applications that answer questions using their own documents, with greater traceability and reduced reliance on unsupported model knowledge.

RAG workflow

Blue: preparing the knowledge source *Purple: answering a question*



Documents > Parsing > Chunking > Embeddings > Vector Database > Retrieval > LLM Context > Answer with Sources

Important: RAG can reduce some hallucinations, but it does not guarantee completely accurate responses. Retrieval can miss relevant passages, and a model can still misread or ignore the context it is given.

A. RAG Fundamentals

TOPICS COVERED

- What RAG is
- Limitations of standalone LLMs
- RAG architecture and workflow
- Retrieval versus generation
- When to use RAG

Why it is taught: To show why standalone LLMs are not enough for document-specific questions and when retrieval is the right design choice.

B. Document Processing

TOPICS COVERED

- Loading PDFs and text files
- Document parsing
- Text cleaning
- Chunking strategies
- Chunk size and overlap
- Metadata extraction

Why it is taught: To prepare source material properly, because retrieval quality depends heavily on how documents are parsed and chunked.

C. Embeddings and Vector Databases

TOPICS COVERED

- What embeddings represent
- Generating text embeddings
- Semantic similarity
- Vector database fundamentals
- Storing and retrieving document chunks

Why it is taught: To enable search by meaning rather than exact keywords, which is the retrieval foundation of RAG.

D. Retrieval Optimization

TOPICS COVERED

- Similarity search
- Top-K retrieval
- Metadata filtering
- Retrieval quality
- Reranking fundamentals
- Context construction

Why it is taught: To improve the relevance of the passages passed to the model, since the answer can only be as good as the retrieved context.

E. RAG Generation and Evaluation

TOPICS COVERED

- Designing RAG prompts
- Supplying retrieved context to an LLM
- Generating grounded answers
- Citations and source references
- Handling missing information
- Evaluating retrieval and answer quality
- Common RAG failure modes
- Practical RAG application workflow

Why it is taught: To produce answers that are grounded in sources, to handle questions the documents cannot answer, and to measure and debug system quality.

4. Hands-On Project Portfolio

Nine practical projects, grouped by technology, let students apply the live curriculum. All are educational prototypes; none is presented as production-ready, deployed or guaranteed to reach a given accuracy.

Group A — ANN Projects

1. Stock Prediction System Using ANN

Technology: Python, Artificial Neural Networks

Use historical stock data to build an ANN-based prediction prototype covering preprocessing, feature preparation, model training and evaluation.

Learning outcome: Understand the full ANN workflow and why proper time-based validation matters for forecasting.

Note: Educational project only, not financial advice. Forecasting has inherent limitations.

2. Indian Currency Note Classifier Using ANN

Technology: Python, ANN, image feature extraction

Build an image classification prototype using suitable image features and an ANN, covering preprocessing, feature extraction, training and evaluation.

Learning outcome: Learn how image data is prepared for a neural network and where a basic ANN approach falls short.

Note: A basic ANN approach may have limitations compared with CNN-based image models.

Group B — CNN and Computer Vision Projects

3. Handwriting Recognition Tool Using CNN

Technology: Python, Convolutional Neural Networks

Develop a CNN-based prototype that recognises handwritten characters or digits, including image preprocessing, model training, prediction and evaluation.

Learning outcome: Apply convolution and pooling concepts to a complete image classification task.

4. Virtual Mouse Using OpenCV

Technology: Python, OpenCV, hand landmark detection

Use computer vision and hand-tracking to control mouse actions through gestures: webcam input, hand landmark detection, gesture interpretation and cursor control.

Learning outcome: Understand real-time computer vision pipelines and how detected landmarks map to actions.

Note: Camera permissions and safe interaction design are important.

Group C — RNN and NLP Projects

5. Automatic Music Generation System Using RNN

Technology: Python, RNN-based sequence models

Train an RNN-based sequence model on suitable musical data and generate new musical sequences.

Learning outcome: Understand sequence learning and generation with recurrent models.

Note: Output quality depends on the dataset and model design.

6. Language Classifier

Technology: Python, NLP

Build an NLP model that predicts the language of a given text, covering text preprocessing, feature representation, model training and evaluation.

Learning outcome: Learn the end-to-end text classification workflow.

Group D — Generative AI and LLM Projects

7. Text-to-SQL Analyst with Guardrails

Technology: LLM API, SQL, sample or sandbox database

Build an application that converts natural-language questions into SQL queries, with schema-aware prompting, query validation, restricted permissions and safe execution.

Learning outcome: Learn to combine LLMs with databases safely, using guardrails and a sandbox rather than an unrestricted production database.

8. Hallucination Highlighter for RAG

Technology: LLM API, RAG, text comparison

Build a prototype that compares generated answers against retrieved source passages and highlights claims that may be unsupported or inconsistent with the provided context.

Learning outcome: Practise evaluating grounded answers and understand the limits of automated checking.

Note: It flags potential issues and does not give a definitive guarantee that an answer is true or false.

9. Code Agent

Technology: LLM API, function-calling fundamentals

Build a basic LLM-powered assistant that helps draft, explain or review code, with task instructions, tool/function-calling basics, controlled execution and error handling.

Learning outcome: Understand how LLMs are connected to tools and why execution must be controlled.

Note: Generated code is not executed without suitable restrictions or human review.

5. Future Learning Pathway — After the 20-Day Program

Want to upskill further after the 20-day program? The next step is **Agentic AI**.

> **Agentic AI is an advanced follow-on course and is not included in the 20-day (40-hour) live curriculum.**

This pathway builds directly on the LLM and RAG knowledge gained in the course, extending applications that answer questions into systems that plan, use tools and act on tasks.

Agentic AI topics

- Introduction to Agentic AI
- AI agents versus conventional LLM applications
- Tool use and function calling
- Planning and task decomposition
- Reasoning and action workflows
- Memory and state management
- Multi-agent workflows
- Agentic RAG
- Human-in-the-loop systems
- Evaluation, safety and guardrails
- Monitoring and deployment fundamentals
- Real-world agentic application development

6. Recorded Learning Resources — Student Portal Access

Students receive portal access to recorded courses so they can revise concepts and learn at their own pace. These are separate resources and are **not part of the 40 hours of live classes**.

A. Python Programming

- Python fundamentals and syntax
- Variables and data types
- Operators and conditional statements
- Loops
- Lists, tuples, sets and dictionaries
- Functions and modules
- File handling
- Exception handling
- Object-oriented programming fundamentals
- Practical Python exercises

B. Advanced Python

- Advanced functions
- Lambda, map, filter and comprehensions
- Iterators and generators
- Decorators
- Object-oriented programming in depth
- Working with files and data formats
- Virtual environments and package management
- Error handling and debugging
- Writing reusable and maintainable code

C. Machine Learning

- Introduction to Machine Learning
- Supervised and unsupervised learning
- Data preprocessing
- Exploratory data analysis
- Feature engineering
- Regression algorithms
- Classification algorithms
- Clustering
- Model evaluation
- Train-test splitting and cross-validation
- Overfitting and underfitting
- Practical model-building workflow

D. SQL

- Introduction to relational databases
- Database and table creation
- SELECT statements
- Filtering and sorting
- Aggregate functions
- GROUP BY and HAVING
- Joins
- Subqueries
- Constraints and keys
- Data manipulation statements
- Practical SQL queries for data analysis